

Bash Scripts And Text Analysis

Today we combine small Linux tools to search, filter, count, and automate a ticket report.

Core skills

- Bash scripts
- pipes
- redirection
- grep
- find
- cut, sort, uniq, wc

Industry connection

- logs
- CSV exports
- test output
- security events
- support tickets
- quick evidence reports

What We Already Know

These commands are the base for today's analysis.

```
$ pwd  
$ ls -la  
$ cd ~/bootcamp  
$ cat notes.txt  
$ python3 hello.py
```

If students are lost, return to pwd and ls.

A Lot Of Technical Evidence Is Plain Text

Linux is strong at turning text into answers quickly.

Examples

- application logs
- server logs
- CSV files
- error reports
- command output
- configuration files

Questions

- what failed?
- how many times?
- which category?
- which files match?
- what changed?
- what should I report?

The Pipeline Mental Model

One command's output can become the next command's input.

tickets.csv

- > tail removes the header
- > cut selects one column
- > sort groups matching values
- > uniq -c counts repeated values
- > sort -nr puts largest first

This is the pipe idea without the syntax.

Never Analyze A File Blind

Look at the top, count the lines, and understand the columns.

```
$ head tickets.csv
```

```
$ wc -l tickets.csv
```

```
$ tail tickets.csv
```

The header row is not a ticket record.

Search For Matching Lines

grep is one of the most useful Linux commands for logs and text files.

```
$ grep "network" tickets.csv
```

```
$ grep -i "password" tickets.csv
```

```
$ grep ",high," tickets.csv
```

grep -i ignores case.

Count Tickets By Category

Build the pipeline one step at a time.

```
$ tail -n +2 tickets.csv | cut -d, -f3
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort | uniq -c
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort | uniq -c | sort -nr
```

Run each stage separately when students are confused.

Save Output As Evidence

Technical work often requires a report, not just a command on screen.

```
$ ./ticket_report.sh > report.txt
```

```
$ cat report.txt
```

```
$ ./ticket_report.sh >> report.txt
```

> overwrites. >> appends.

Save The Workflow

A script makes the analysis repeatable.

```
#!/usr/bin/env bash
set -euo pipefail
```

```
DATA_FILE="${1:-tickets.csv}"
echo "Tickets by category:"
tail -n +2 "$DATA_FILE" | cut -d, -f3 | sort | uniq -c | sort -nr
```

Scripts are repeatable command recipes.

Locate Files By Pattern

find searches through folders for matching files.

```
$ find . -name "*.sh"
```

```
$ find . -name "*.txt"
```

```
$ find ~/bootcamp -name "*.py"
```

The dot means start in the current folder.

Commands Are Not The Final Answer

Students need to translate output into a sentence.

Output

- 3 network
- 2 software
- 2 password
- 2 access
- 1 hardware

Explanation

- network is the largest category
- there are 10 ticket records
- four tickets are high priority
- the report is evidence

Pipeline Debugging

Fix one piece at a time.

Typical issues

- file not found
- wrong folder
- missing quotes
- wrong field number
- spaces in copied command
- script not executable

Troubleshoot

- pwd
- ls
- head tickets.csv
- run one command stage
- use bash script.sh
- then try ./script.sh

Brightspace Submission

Students submit both command evidence and writing.

Evidence

- head output
- line count
- grep results
- category pipeline
- ticket_report.sh
- report.txt output

Writing

- what grep does
- what a pipe does
- most common category
- why find is useful
- job connection reflection

What Success Looks Like Today

Students can turn a text file into a basic report.

Students should be able to say

- I can search with grep
- I can explain a pipe
- I can count categories
- I can save output
- I can run a Bash script

Next class

- Python CSV reading
- SQLite database file
- tables and rows
- SQL queries
- persistence